

In[310]:=

```
Print["Wolfram Language ready!"]
Print[$Version]
```

Wolfram Language ready!

14.3.0 for Linux x86 (64-bit) (July 8, 2025)

In[312]:=

```
(* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *)
(* *)
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *)
(* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *)
(* *)
(* When V[t] ≥ Vthresh, neuron spikes and resets *)
```

In[313]:=

In[314]:=

```
(* ===== Neuron Parameters ===== *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)
(* ===== Input Current ===== *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t && t ≤ 0.4, 18.0, 0.0];
Print["Parameters defined."]
Parameters defined.
KNOWLEDGE CHECK
```

Answer these in a text cell below your code.

1. The input current is defined as a function of time using If[]. In plain English, what does Iinput[t] return at t = 0.03? At t = 0.2? At t = 0.5?

The input current function Iinput[t] returns 0.0 at t = 0.03 because that time is before the current turns on. It returns 18.0 at t = 0.2 because that time is inside the active current interval. It returns 0.0 at t = 0.5 because that time is after the current turns off.

2. Why do we use := (delayed assignment) for Iinput instead of = (immediate assignment)?

We use $:=$ for input because it is a delayed assignment, which means Wolfram evaluates the function each time t is used. This is important because the function depends on the current value of t . If $=$ were used, it would try to evaluate immediately instead of waiting for a specific input value.

Solve the LIF Equation

In[321]:=

```
Clear[V, sol, spikeTimes, spikeCount];
```

```
result = Reap[
  NDSolveValue[
    {
      tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
      V[0] == Vrest,
      WhenEvent[
        V[t] ≥ Vthresh - 0.001,
        {Sow[t], V[t] → Vreset}
      ]
    },
    V,
    {t, 0, 0.5},
    MaxStepSize → 0.0001
  ]
];
```

```
sol = result[[1]];
spikeTimes = Flatten[result[[2]]];
spikeCount = Length[spikeTimes];
```

```
Print["Neuron fired ", spikeCount, " spikes"];
```

```
Print["Spike times: ", spikeTimes];
```

```
Neuron fired 8 spikes
```

```
Spike times:
```

```
{0.0858285, 0.12656, 0.16729, 0.208021, 0.248752, 0.289483, 0.330214, 0.370945}
```

Plot the Membrane Potential

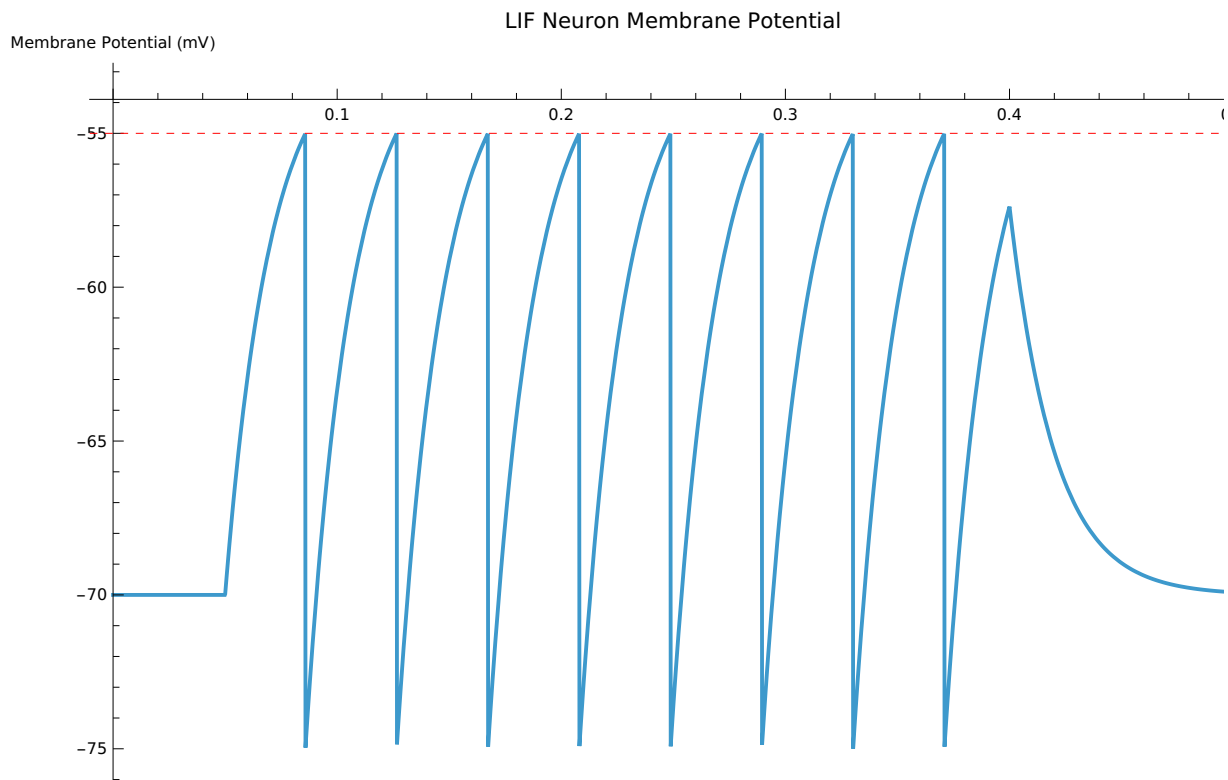
In[328]:=

```

Plot[
  sol[t],
  {t, 0, 0.5},
  PlotRange → All,
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → "LIF Neuron Membrane Potential",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
]

```

Out[328]=



KNOWLEDGE CHECK

1. Look at the shape of V between spikes. Is it a straight line or a curve? Why?

The shape of V between spikes is a curve, not a straight line. This happens because the membrane potential changes according to a differential equation, so it rises gradually toward threshold in an exponential way.

2. What happens to V after the current turns off ($t > 0.4$)? Describe what you see and explain it in terms of the LIF equation.

After the current turns off at $t > 0.4$, the membrane potential falls back toward the resting potential.

This happens because once the input current is removed, only the leak part of the LIF equation remains, which pulls the voltage back down toward V_{rest} .

Part B: Interactive Manipulate[] Dashboard

Build the Dashboard

In[331]:=

```
Manipulate[
Module[{result, sol, sc, st = {}, iFunc},

(* Local input current using slider-controlled amplitude *)
iFunc[t_] := If[0.05 ≤ t && t ≤ 0.4, currentVal, 0.0];

(* Use the same Reap/Sow pattern that worked in Part A *)
result = Reap[
NDSolveValue[
{
tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
V[0] == Vrest,
WhenEvent[
V[t] ≥ thresh - 0.001,
{Sow[t], V[t] → Vreset}
]
},
V,
{t, 0, 0.5},
MaxStepSize → 0.0001
]
];

sol = result[[1]];
st = Flatten[result[[2]]];
sc = Length[st];

Plot[
sol[t],
{t, 0, 0.5},
PlotRange → All,
AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
PlotLabel → StringForm["Spikes: ``", sc],
Epilog → {
Red, Dashed,
```

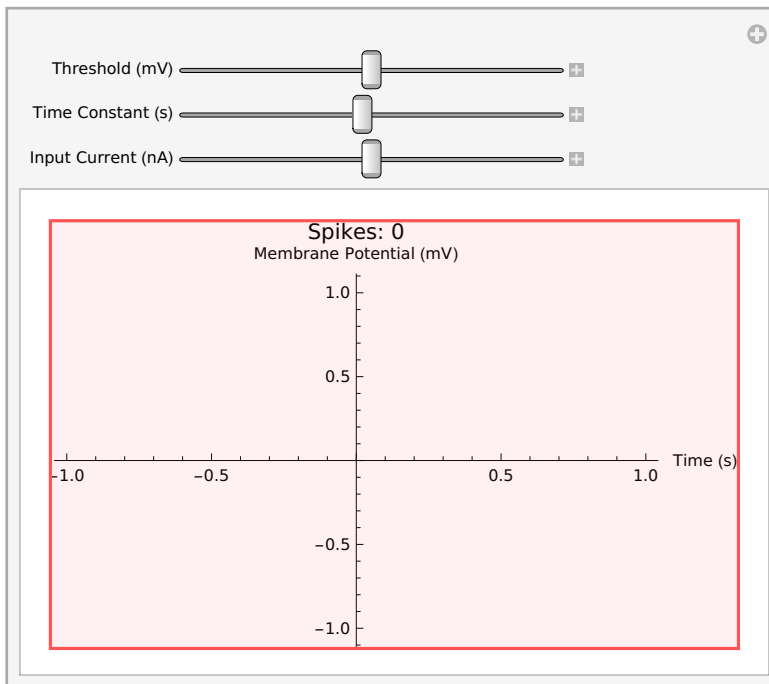
```

    InfiniteLine[{{0, thresh}, {1, thresh}}]
  }
]
],

{{thresh, -55.0, "Threshold (mV)", -65, -45, 1},
{{tauVal, 0.020, "Time Constant (s)", 0.005, 0.100, 0.005},
{{currentVal, 18.0, "Input Current (nA)", 0, 40, 1}
]

```

Out[331]=



KNOWLEDGE CHECK

1. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?

Using the dashboard, the minimum input current needed to make the neuron fire at least one spike was about 15 nA. This is the rheobase, which is the smallest current that can push the membrane potential to threshold.

2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?

With the current set to 25 nA, the neuron stopped firing at about -45 mV. At this setting, the neuron

only fired 1 spike, which shows that raising the threshold makes it much harder for the membrane potential to reach the spike level. This happens because the voltage has to climb higher before a spike can occur.

3. Set threshold to -55 and current to 20. Compare $\tau = 0.010$ vs $\tau = 0.050$. Which produces more spikes? Which produces a smoother membrane potential curve?

With threshold set to -55 and current set to 20, $\tau = 0.010$ produced 21 spikes, while $\tau = 0.050$ produced only 4 spikes. The smaller τ produced more spikes because the membrane potential changed faster and reached threshold more often. The larger τ produced fewer spikes and a smoother membrane potential curve because the voltage changed more slowly over time.

REFLECTION

The Manipulate[] dashboard lets you explore neuron behavior interactively in a way that static code cannot. Describe one insight you gained from adjusting the sliders that you did NOT get from reading the booklet. How did the visual feedback change your understanding?

One thing I learned from the interactive dashboard was the importance of the neuron parameters in the firing behavior. For instance, when I set the threshold to -45 mV with the current set to 25 nA, the neuron fired only 1 time. This illustrates the effect of the increased threshold in almost inhibiting the firing of the neuron. I also saw the effect of the value of τ on the number of spikes fired by the neuron. For instance, when I set τ to 0.010 and 0.050 with the same threshold and current, the neuron with $\tau = 0.010$ fired 21 spikes, whereas the neuron with $\tau = 0.050$ fired only 4 spikes.

Part C: Firing Rate Curve (f-I Curve)

Generate the f-I Curve

In[336]:=

```

fiData = Table[
Module[{result, solLocal, sc, st = {}},

result = Reap[
NDSolveValue[
{
tau*Vfi'[t] == -(Vfi[t] - Vrest) +
R*If[0.05 ≤ t && t ≤ 0.4, Icurr, 0.0],
Vfi[0] == Vrest,
WhenEvent[
Vfi[t] ≥ Vthresh - 0.001,
{Sow[t], Vfi[t] → Vreset}
]
},
Vfi,
{t, 0, 0.5},
MaxStepSize → 0.0001
]
];

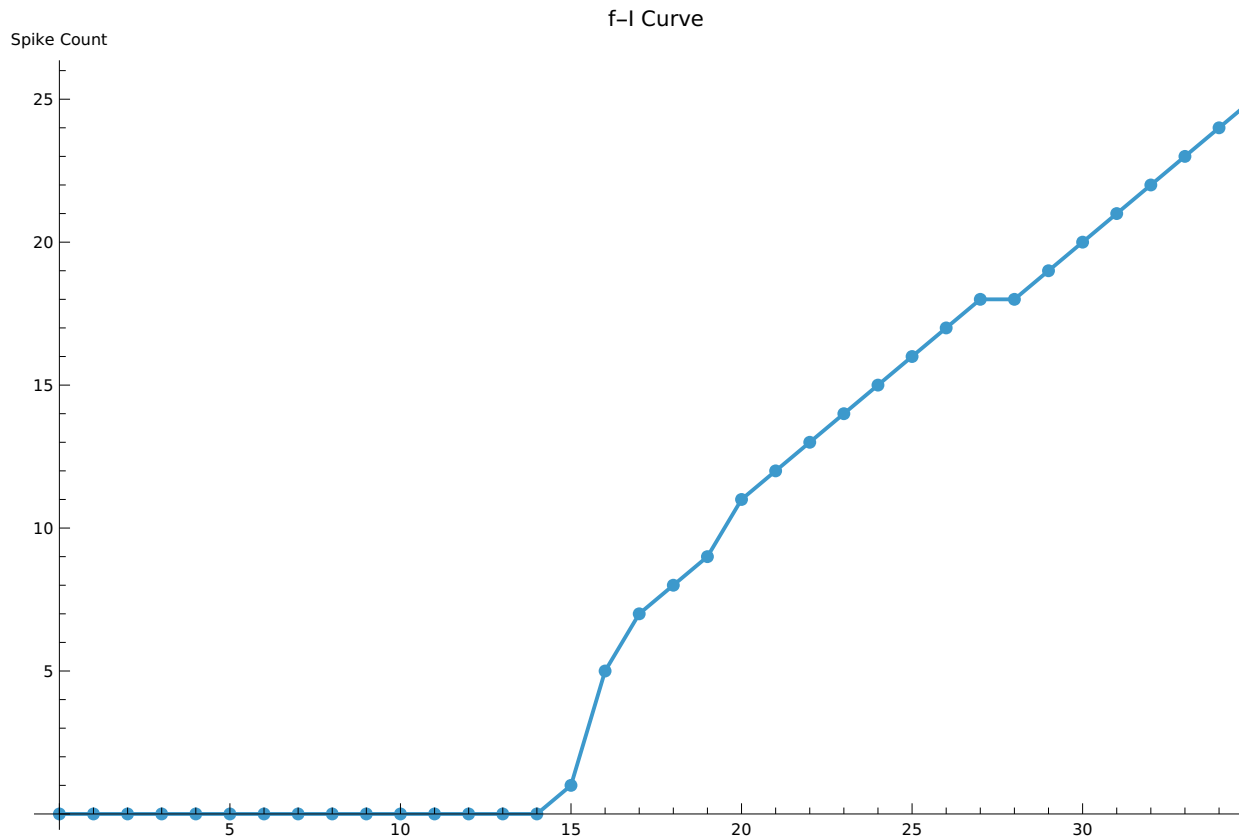
solLocal = result[[1]];
st = Flatten[result[[2]]];
sc = Length[st];

{Icurr, sc}
],
{Icurr, 0, 35, 1}
];

ListLinePlot[
fiData,
AxesLabel → {"Input Current (nA)", "Spike Count"},
PlotLabel → "f-I Curve",
PlotMarkers → Automatic
]

```

Out[337]=



KNOWLEDGE CHECK

1. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.

The neuron's firing began at around 15 nA. This is the same rheobase that I calculated using the interactive dashboard earlier in the lab.

2. Is the f-I curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?

The curve does not perfectly resemble a straight line. It starts off flat, with no action potential occurring at low currents, and then starts to increase as the currents increase to the point where they are strong enough to reach the threshold. A perfectly straight line would mean that the neuron fires in direct proportion to the strength of the stimulus.

3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?

The LIF model does not have a strong saturation effect at high currents. The spike count continues to increase as the currents increase in strength, but in the real world, there are many factors at play, such as the strength of the refractory period.

Part D: Encoding a Signal as Spikes

Rate Coding with a Varying Signal

In[338]:=

```
Isine[t_] := 15.0 + 12.0*Sin[2 Pi*5*t];
```

```
resultSine = Reap[
  NDSolveValue[
    {
      tau*Vsine'[t] == -(Vsine[t] - Vrest) + R*Isine[t],
      Vsine[0] == Vrest,
      WhenEvent[
        Vsine[t] ≥ Vthresh - 0.001,
        {Sow[t], Vsine[t] → Vreset}
      ]
    },
    Vsine,
    {t, 0, 1.0},
    MaxStepSize → 0.0001
  ]
];
```

```
solSine = resultSine[[1]];
sineSpikeTimes = Flatten[resultSine[[2]]];
sineSpikeCount = Length[sineSpikeTimes];
```

```
Print["Neuron fired ", sineSpikeCount, " spikes"];
Print["Spike times: ", sineSpikeTimes];
```

```
Neuron fired 19 spikes
```

```
Spike times: {0.0260969, 0.0468147, 0.0671062, 0.217904, 0.240205,
  0.260032, 0.284429, 0.417967, 0.440253, 0.460081, 0.48451, 0.617968,
  0.640253, 0.660081, 0.68451, 0.817968, 0.840253, 0.860081, 0.88451}
```

Create the Two-Panel Plot

In[345]:=

```

plot1 = Plot[
  solSine[t],
  {t, 0, 1.0},
  PlotRange → All,
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → "Membrane Potential",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
];

```

```

plot2 = Plot[
  Isine[t],
  {t, 0, 1.0},
  PlotRange → All,
  AxesLabel → {"Time (s)", "Input Current (nA)"},
  PlotLabel → "Sine Wave Input Current"
];

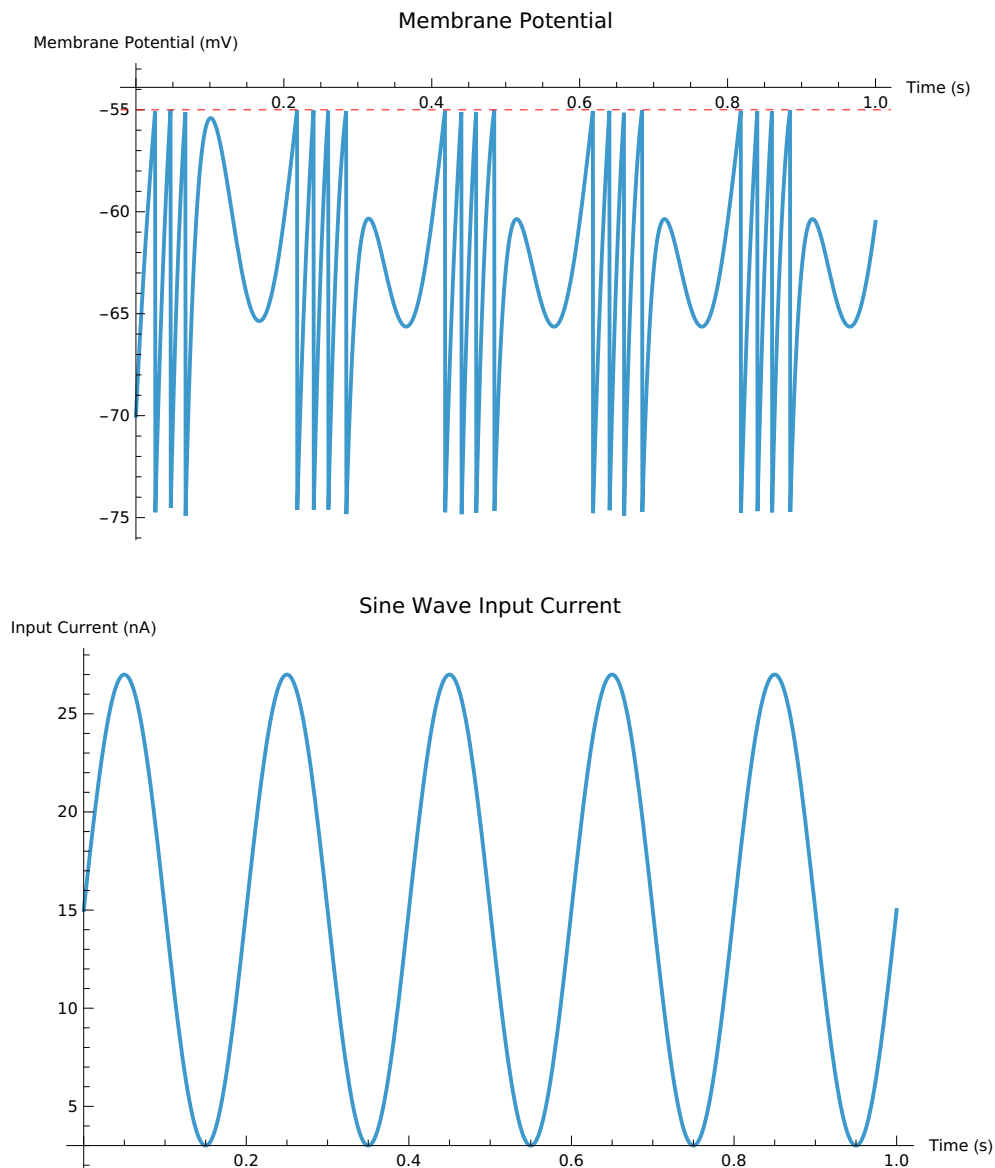
```

```

GraphicsColumn[{plot1, plot2}]

```

Out[347]=



REFLECTION

Look at your spike train alongside the sine wave input. The neuron is acting as an analog-to-digital converter: it takes a continuous signal and converts it into discrete spikes. How well does the spike pattern represent the original signal? What information is preserved? What is lost?

When I compared the spike train with the sine wave input signal, I observed that the neuron fired spikes more frequently at the peaks of the input signal and less frequently at the troughs of the input signal. It appears that the neuron fires more vigorously at higher input signal values and less vigorously at lower input signal values.

The spike train encodes the shape of the signal because the density of the spikes mirrors the shape of the sine wave. When the sine wave is increasing, the neuron fires off more spikes, and when the sine wave is decreasing, the neuron fires off fewer spikes. This is an example of how a neuron encodes a signal.

The information that is preserved in the encoding is the timing of when the signal is strong or weak, and the rhythm of the sine wave. The information that is lost in the encoding is the smooth shape of the signal, which is not present in the spike train. This is an example of how a neuron acts like an analog-to-digital converter, taking a continuous signal and converting it into a spike train that still has important information about the original signal.

Part E: Experiments

Experiment 1: Threshold Sweep

In[348]:=

```

thresholdData = Table[
  Module[{result, solLocal, sc, st = {}},

    result = Reap[
      NDSolveValue[
        {
          tau*Vth'[t] == -(Vth[t] - Vrest) + R*Iinput[t],
          Vth[0] == Vrest,
          WhenEvent[
            Vth[t] ≥ th - 0.001,
            {Sow[t], Vth[t] → Vreset}
          ]
        },
        Vth,
        {t, 0, 0.5},
        MaxStepSize → 0.0001
      ]
    ];

    solLocal = result[[1]];
    st = Flatten[result[[2]]];
    sc = Length[st];

    {th, sc}
  ],
  {th, -65, -45, 5}
];

```

thresholdData

```

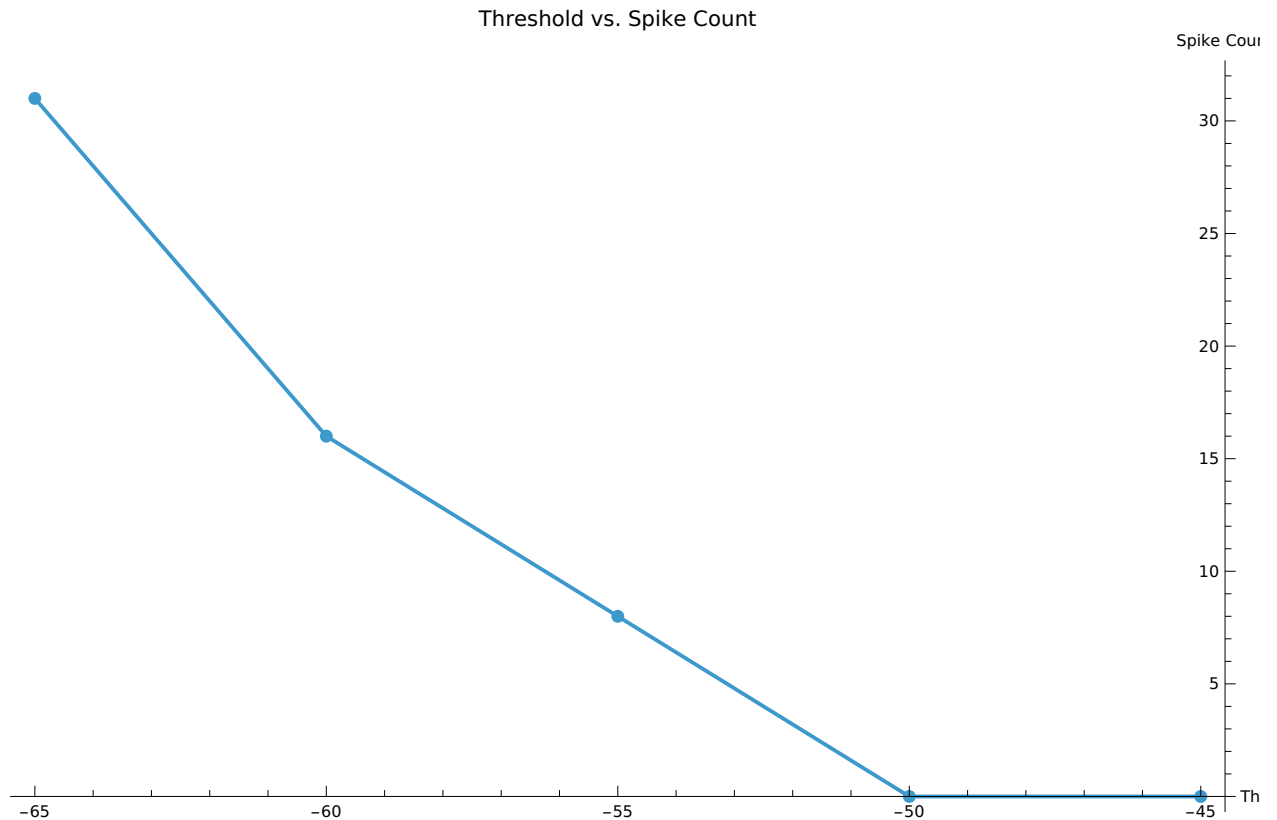
ListLinePlot[
  thresholdData,
  AxesLabel → {"Threshold (mV)", "Spike Count"},
  PlotLabel → "Threshold vs. Spike Count",
  PlotMarkers → Automatic
]

```

Out[349]=

```
{{-65, 31}, {-60, 16}, {-55, 8}, {-50, 0}, {-45, 0}}
```

Out[350]=



1. What parameter did you change, and to what values?

I changed the threshold from -65 mV to -45 mV in steps of 5 mV.

2. What did you expect to happen?

I expected that lower thresholds would produce more spikes because the membrane potential would reach the spike level more easily. I also expected that higher thresholds would reduce the number of spikes.

3. What actually happened? (Include the output or screenshot.)

The results matched my expectation. When the threshold was set to -65 mV, the neuron fired 31 spikes. At -60 mV it fired 16 spikes, and at -55 mV it fired 8 spikes. When the threshold was increased to -50 mV and -45 mV, the neuron produced 0 spikes.

```
{{-65,31},{-60,16},{-55,8},{-50,0},{-45,0}}
```

4. Why does this make sense given the LIF equation?

This makes sense because, according to the LIF equation, when you input current into the neuron, it will start rising up toward a certain value that is dependent on that current. If you set it too low, it will cross that threshold many times, so it will have many spikes. If you set it too high, it will never cross that threshold, so it will have no spikes.

Experiment 2: Time Constant Comparison

In[351]:=

```

makePlot[tauTest_] := Module[{result, solLocal, sc, st = {}},

  result = Reap[
    NDSolveValue[
      {
        tauTest*Vtau'[t] == -(Vtau[t] - Vrest) + R*If[0.05 ≤ t && t ≤ 0.4, 20, 0.0],
        Vtau[0] == Vrest,
        WhenEvent[
          Vtau[t] ≥ -55 - 0.001,
          {Sow[t], Vtau[t] → Vreset}
        ]
      },
      Vtau,
      {t, 0, 0.5},
      MaxStepSize → 0.0001
    ]
  ];

  solLocal = result[[1]];
  st = Flatten[result[[2]]];
  sc = Length[st];

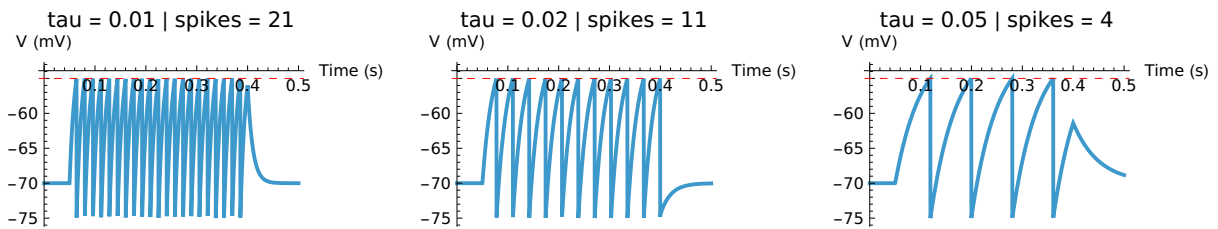
  Plot[
    solLocal[t],
    {t, 0, 0.5},
    PlotRange → All,
    AxesLabel → {"Time (s)", "V (mV)"},
    PlotLabel → StringForm["tau = `` | spikes = ``", tauTest, sc],
    Epilog → {Red, Dashed, InfiniteLine[{{0, -55}, {1, -55}}]}
  ]
];

plotA = makePlot[0.010];
plotB = makePlot[0.020];
plotC = makePlot[0.050];

GraphicsRow[{plotA, plotB, plotC}]

```


Out[355]=



1. What parameter did you change, and to what values?

In this experiment, I changed the membrane time constant, which is represented by τ . I changed it to three different values: 0.01, 0.02, and 0.05.

2. What did you expect to happen?

In this experiment, I expected that when I decreased the value of τ , it would allow the membrane potential to change more quickly, increasing the number of spikes. I also expected that when I increased the value of τ , it would decrease the number of spikes.

What actually happened? (Include the output or screenshot.)

I got results that matched my expectations. For $\tau = 0.01$, the neuron fired 21 spikes. For $\tau = 0.02$, the neuron fired 11 spikes. For $\tau = 0.05$, the neuron fired only 4 spikes. Also, the membrane potential was smoother and slower for higher values of τ .

4. Why does this make sense given the LIF equation?

This makes sense because, as we saw in the LIF equation, τ determines how fast the membrane potential changes in response to an input current. A smaller τ means that the membrane potential changes more quickly, so that the neuron can fire more often. A higher τ means that the change in voltage is slower, so that it takes longer for the voltage to reach threshold, resulting in fewer firing spikes.

Experiment 3: Noisy Input

In[356]:=

```

runNoisy[] := Module[{result, solLocal, sc, st = {}, Inoisy},

  Inoisy[t_] := 18.0 + RandomReal[{-8, 8}];

  result = Reap[
    NDSolveValue[
      {
        tau*Vn'[t] == -(Vn[t] - Vrest) + R*Inoisy[t],
        Vn[0] == Vrest,
        WhenEvent[
          Vn[t] ≥ Vthresh - 0.001,
          {Sow[t], Vn[t] → Vreset}
        ]
      },
      Vn,
      {t, 0, 0.5},
      MaxStepSize → 0.0001
    ]
  ];

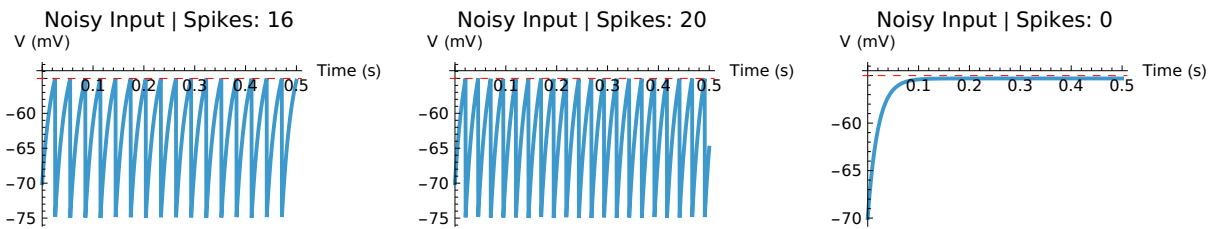
  solLocal = result[[1]];
  st = Flatten[result[[2]]];
  sc = Length[st];

  Plot[
    solLocal[t],
    {t, 0, 0.5},
    PlotRange → All,
    AxesLabel → {"Time (s)", "V (mV)"},
    PlotLabel → StringForm["Noisy Input | Spikes: ``", sc],
    Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
  ];

GraphicsRow[{runNoisy[], runNoisy[], runNoisy[]}]

```

Out[357]=



1. What parameter did you change, and to what values?

In this experiment, I changed the constant input current to a noisy input current, represented by the code `18.0 + RandomReal[{-8, 8}]`. Therefore, the input current was varying randomly about 18 nA.

2. What did you expect to happen?

I expected that the spike pattern would change from run to run, as the input current was no longer constant. Since the input current was varying randomly, I expected that the membrane potential should sometimes reach threshold more often and sometimes less often.

3. What actually happened? (Include the output or screenshot.)

The results were as expected. The simulation had a different spike pattern each time. In the simulation, the neuron had 16 spikes in the first simulation, 20 spikes in the second simulation, and 0 spikes in the third simulation.

4. Why does this make sense with regard to the LIF equation?

The above makes sense because the LIF equation depends only on the input current. When the input current varies at random, the membrane potential varies at random too. When the input current remains high enough for a certain period, the membrane potential fires the spikes. When the input current remains low, the membrane potential remains low and does not reach the threshold to generate the spike, resulting in fewer or no spikes at all.

Part F: Final Reflection

REFLECTION

1. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this threshold behavior tell you about how biological neurons make “decisions”?

By using the Manipulate dashboard, I found that if the current level was less than 15 nA, the neuron

did not fire any spike. As the current level increased to a level above 15 nA, the neuron fired a spike. As the current level continued to increase, the rate of firing of the neuron also increased. This shows the threshold behavior of the neuron, where the neuron will not fire any spike before the membrane potential reaches a critical level. This is similar to the behavior of biological neurons in deciding to fire an action potential.

2. Examine your f - I curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

The f - I curve was not perfectly linear. At low current values, it was flat, which meant that the neuron was not firing. Once it reached the rheobase current value approximately 15 nA, the spike count started increasing as the current was increased. This is biologically relevant because it indicates that neurons can convey information about the strength of a stimulus by increasing their firing rates accordingly.

3. How does changing the membrane time constant affect firing? In biological terms, what types of neurons might have short vs. long time constants, and what would each be good at?

The change in the membrane time constant had a significant impact on the firing behavior. For a smaller time constant ($\tau = 0.01$), the neuron fired many spikes due to the quick change in the membrane potential in response to the input current. For a larger time constant ($\tau = 0.05$), the neuron fired fewer spikes due to the slow change in the membrane potential. For biological neurons, a smaller time constant can help the neuron detect changes in the input signal, whereas a larger time constant can help the neuron integrate the signal over a longer period.

4. Compare the LIF model to what you learned about real neurons in the booklet. What does the model capture well? What important features does it miss? (Think about: dendrite morphology, multiple ion channels, neurotransmitter dynamics, adaptation.)

The LIF model has also been able to incorporate many other key characteristics of real neurons, including their membrane potential, threshold for firing, spike generation, and reset following a spike. These characteristics can be considered an extension of the general properties of action potentials in real neurons. However, it should also be considered that even though the LIF model has been able to incorporate many characteristics of real neurons, it is still considered a simplification of real neurons, which have many other complex characteristics, including their dendritic structure, different types of ion channels, and other characteristics such as spike adaptation.

5. Wolfram's Manipulate[] gave you real-time interactive exploration. How did this change your understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

The use of Wolfram's Manipulate function made the neuron model much easier to comprehend compared to simply reading about it. The ability to change the sliders and instantaneously observe the effects of the changes to the spike behaviors helped to make the concepts much clearer. The most surprising aspect of the neuron model that I discovered is the sensitivity of the neuron to changes in the parameters. For instance, a slight change in the threshold and time constant could have a significant impact on the number of spikes.